

VI. Desarrollo del Sistema

Cacao Leaf - Aplicacion movil para el analisis preliminar de hojas de cacao

Sintesis

El sistema implementa una aplicacion movil con React Native y Expo conectada a un backend Django REST Framework. Permite capturar o seleccionar una imagen, enviarla mediante API, procesarla, guardar el resultado en SQLite, consultar historial, abrir detalle y limpiar registros con confirmacion.

6.1. Descripcion general del sistema

Cacao Leaf es un prototipo funcional orientado a apoyar la clasificacion preliminar de sintomas visibles en hojas de cacao. El sistema se compone de una aplicacion movil, un servicio API y una base de datos local. La aplicacion no entrega un diagnostico definitivo; muestra una orientacion inicial, nivel de confianza, observacion y recomendacion para que el usuario pueda registrar evidencia y decidir si necesita una revision especializada.

La solucion fue construida priorizando criterios de IHC: lenguaje comprensible, flujo directo, retroalimentacion visual, prevencion de errores y control del usuario sobre el historial. Por ello, las pantallas evitan terminos tecnicos como backend, dataset o modelo entrenado en la interfaz final.

6.2. Arquitectura del sistema

Arquitectura del sistema Cacao Leaf



Principio de diseno: el usuario ve lenguaje simple; la complejidad tecnica queda oculta en la API.

La arquitectura sigue un esquema cliente-servidor. La aplicacion Expo actua como cliente, Django REST Framework expone los endpoints y SQLite almacena los registros de analisis. Las imagenes se guardan como archivos de media y la base de datos conserva la ruta, estado, confianza, notas, recomendacion y fecha.

6.3. Stack tecnologico

Capa	Tecnologia	Uso en el proyecto
Frontend	React Native + Expo	Construccion de la app movil y ejecucion web para demo con las pantallas Inicio, Analizar, Historial, Detalle e Info.

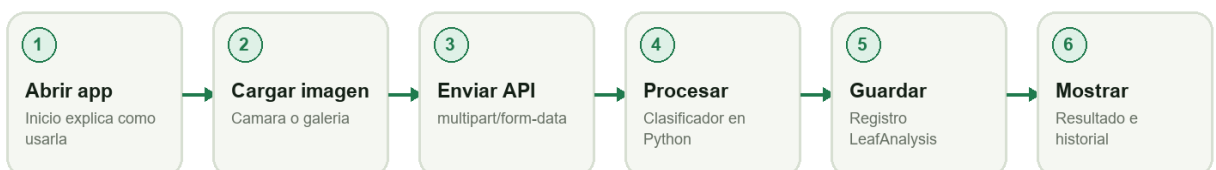
Capa	Tecnologia	Uso en el proyecto
Backend	Django + Django REST Framework	API REST para recibir imagenes, validar archivos, ejecutar clasificacion y devolver respuestas JSON.
Base de datos	SQLite	Persistencia local de analisis realizados durante la prueba del prototipo.
Procesamiento	Python + Pillow + modulo diagnostics	Validacion de imagenes y clasificacion preliminar de hoja sana o posible patologia visible.
Iconografia/UI	lucide-react-native	Iconos para acciones de camara, galeria, historial, alerta, confirmacion e informacion.
Pruebas	Django TestCase + TypeScript	Validacion automatica del backend y comprobacion de tipos del frontend.

6.4. Modulos implementados

Modulo	Archivo o carpeta	Funcion
Pantalla Inicio	mobile/src/screens/HomeScreen.tsx	Presenta Cacao Leaf, explica como usar la app y dirige al analisis.
Pantalla Analizar	mobile/src/screens/AnalyzeScreen.tsx	Permite usar camara, galeria, previsualizar imagen, procesar y mostrar resultado.
Pantalla Historial	mobile/src/screens/HistoryScreen.tsx	Lista registros, abre detalle ampliado y permite limpiar historial con confirmacion.
Pantalla Info	mobile/src/screens/InfoScreen.tsx	Explica senales visuales, limitaciones y uso recomendado.
Cliente API	mobile/src/api/client.ts	Consume endpoints de Django para crear, listar y limpiar analisis.
Modelo de datos	backend/diagnostics/models.py	Define LeafAnalysis con imagen, estado, confianza, etiqueta, notas y fecha.
Serializador	backend/diagnostics/serializers.py	Valida imagenes, ejecuta clasificacion y arma contrato JSON.
Vistas/API	backend/diagnostics/views.py	Expone health check, creacion, listado, detalle y limpieza de historial.
Clasificador	backend/diagnostics/classifier.py	Punto central para procesar la imagen y devolver ClassificationResult.

6.5. Flujo funcional

Flujo funcional implementado



Alcance: clasificacion preliminar. No reemplaza revision de un especialista agricola.

Paso	Descripcion	Evidencia tecnica
1	El usuario abre la app y accede a Analizar.	TabBar + HomeScreen
2	Selecciona imagen desde camara o galeria.	expo-image-picker
3	La app arma FormData con la imagen.	createAnalysis(imageUri)
4	La API recibe POST /api/analyses/.	LeafAnalysisViewSet.create
5	El serializador valida formato y tamano.	validate_image
6	El clasificador genera resultado preliminar.	classify_cacao_leaf
7	Se guarda registro en SQLite.	LeafAnalysis.objects.create
8	La app muestra resultado e historial.	AnalyzeScreen + HistoryScreen

6.6. Base de datos

La persistencia se implementa con SQLite mediante el modelo LeafAnalysis. El modelo representa cada analisis realizado por el usuario y permite ordenar los registros desde el mas reciente. No se implemento login en el MVP porque el objetivo de IHC es validar primero el flujo principal con minima friccion; la autentificacion queda como mejora futura si se requiere historial privado por usuario.

Campo	Tipo	Descripcion
id	AutoField	Identificador unico del analisis.
image	ImageField	Archivo subido en media/leaf_uploads/.
status	CharField	healthy o pathology.
confidence	DecimalField	Porcentaje de confianza con dos decimales.
disease_label	CharField	Etiqueta visible para el usuario.
recommendation	TextField	Recomendacion orientada al usuario final.
notes	TextField	Observacion del analisis.
created_at	DateTimeField	Fecha y hora de creacion.

6.7. Endpoints principales

Metodo	Ruta	Funcion	Respuesta esperada
GET	/api/health/	Verifica disponibilidad del servicio.	{"status": "ok"}
GET	/api/analyses/	Lista historial de analisis.	Arreglo JSON ordenado por fecha descendente.
POST	/api/analyses/	Recibe imagen multipart y crea analisis.	Objeto JSON con estado, confianza, notas y recomendacion.
GET	/api/analyses/{id}/	Obtiene detalle de un analisis.	Objeto JSON de LeafAnalysis.
DELETE	/api/analyses/clear/	Elimina historial completo e imagenes asociadas.	{"deleted": cantidad}

6.8. Contrato JSON

```
{ "id": 1, "image_url": "http://127.0.0.1:8000/media/leaf_uploads/cacao-leaf.jpg", "status": "pathology", "status_display": "Posible patologia", "confidence": "88.97", "disease_label": "Posible patologia" }
```

```
visible", "recommendation": "Revisa la planta con mayor detalle y consulta a un especialista agricola...",
"notes": "La imagen presenta senales visuales que podrian estar asociadas a una patologia.", "created_at":
"2026-06-12T12:36:09Z" }
```

6.9. Validaciones y manejo de errores

Validacion	Implementacion	Beneficio IHC
Formato permitido	Solo JPG, PNG o WEBP.	Evita errores por archivos incompatibles.
Tamano maximo	5 MB.	Reduce fallos por archivos demasiado pesados.
Verificacion real de imagen	Pillow abre y verifica el archivo.	Evita aceptar archivos falsos o corruptos.
Boton Procesar condicionado	Se desactiva si no hay imagen.	Previene acciones invalidas.
Mensajes comprensibles	Errores como 'No se pudo procesar la imagen'.	Ayuda al usuario a recuperarse.
Confirmacion de limpieza	Modal antes de borrar historial.	Evita perdida accidental de registros.

6.10. Interfaz desarrollada

El frontend mantiene una navegacion inferior con cuatro secciones: Inicio, Analizar, Historial e Info. Esta decision favorece reconocimiento antes que memoria, porque las acciones principales permanecen visibles durante la experiencia.

Pantalla	Elementos principales	Criterio IHC aplicado
Inicio	Imagen de hoja, titulo, boton Analizar hoja y pasos de uso.	Comprension rapida del proposito.
Analizar	Camara, Galeria, Procesar imagen, chips de calidad de foto.	Prevencion de errores y claridad.
Resultado	Estado, confianza, observacion y recomendacion.	Retroalimentacion visible.
Historial	Cards con imagen, fecha, confianza y Ver detalle.	Seguimiento y eficiencia.
Detalle	Imagen grande, resultado, fecha, observacion y recomendacion.	Informacion ampliada sin saturar la lista.
Confirmacion	Cancelar o Limpiar.	Control y libertad del usuario.
Info	Senales visuales, limitacion y uso recomendado.	Ayuda y documentacion.

6.11. Pruebas tecnicas realizadas

Prueba	Comando o validacion	Resultado
Backend checks	python manage.py check	Sin problemas del sistema.
Backend tests	python manage.py test	7 pruebas ejecutadas correctamente.
Frontend typecheck	npm run typecheck	Sin errores TypeScript.
Health endpoint	GET /api/health/	Devuelve status ok.
Subida de imagen	POST /api/analyses/	Crea analisis y devuelve JSON.
Historial	GET /api/analyses/	Lista registros guardados.
Limpiar historial	DELETE /api/analyses/clear/	Elimina registros e imagenes asociadas.

6.12. Instalacion y ejecucion

Backend

```
cd D:\xampp\htdocs\entregable2_IHC\backend .\venv\Scripts\Activate.ps1 python manage.py migrate python manage.py runserver 0.0.0.0:8000
```

Frontend web para demo

```
cd D:\xampp\htdocs\entregable2_IHC\mobile npm.cmd install npm.cmd run web
```

Movil fisico con Expo Go

\$env:EXPO_PUBLIC_API_URL="http://IP_DE_LA_LAPTOP:8000" npm.cmd run start El celular debe estar conectado a la misma red que la laptop y abrir el QR con Expo Go.

6.13. Seguridad, privacidad y alcance

Aspecto	Decision actual	Mejora futura
Autenticacion	No se incluye login en el MVP para reducir friccion y validar el flujo principal.	Agregar cuentas si se requiere historial privado.
Datos personales	No se solicitan nombres, ubicacion ni datos de finca.	Definir politica de privacidad si se recopilan datos reales.
Imagenes	Se almacenan localmente en media uploads para demo.	Usar almacenamiento seguro en produccion.
Diagnostico	Se comunica como resultado preliminar.	Validacion con especialista agricola y dataset ampliado.
Despliegue	Ejecucion local para exposicion.	Servidor backend y build movil si pasa a produccion.

6.14. Relacion con los objetivos del proyecto

Objetivo	Evidencia de cumplimiento
Permitir registro visual de una hoja.	Camara y galeria en AnalyzeScreen.
Procesar imagen y entregar resultado.	POST /api/analyses/ + classify_cacao_leaf.
Mostrar informacion comprensible.	Resultado con estado, confianza, observacion y recomendacion.
Guardar seguimiento.	Modelo LeafAnalysis e historial.
Dar control al usuario.	Detalle de analisis y limpiar historial con confirmacion.
Aplicar criterios IHC.	Lenguaje no tecnico, prevencion de errores, ayuda y navegacion visible.

6.15. Conclusiones del desarrollo

- El sistema cumple con frontend funcional, backend funcional, integracion API y persistencia local.
- La app puede ejecutarse como movil mediante Expo Go y como web para demostracion con Expo Web.
- El historial, detalle y limpieza de registros evidencian una evolucion del MVP hacia una experiencia mas completa.
- La arquitectura permite reemplazar o mejorar el clasificador sin romper el contrato de la app.
- Para produccion, se recomienda agregar autenticacion, almacenamiento seguro, despliegue backend y validacion experta del modelo.